A Streaming BF16 Self-Attention Accelerator with RoPE, GQA, and a Time-Shared Systolic MAC Array

İTÜ YONGA — Yonga Attention Engine (YAE) Project Team
Department of Electronics and Communication Engineering, Istanbul Technical University, Istanbul, Turkey

***Abstract*—** This report presents the design of a SystemVerilog RTL accelerator that computes the self-attention block of transformer models in hardware. The core implements a streaming, AXI-Stream-based datapath in bfloat16 (BF16) arithmetic, and integrates Rotary Positional Encoding (RoPE), Grouped Query Attention (GQA), and a time-shared systolic processing-element (PE) array that performs both the $Q \cdot K^T$ similarity computation and the score·V reduction on the same physical hardware. Resource efficiency is achieved through ping-pong double buffering, valid/ready handshaking throughout, and arbitrated sharing of the most expensive multiply-accumulate (MAC) resource. The design currently comprises more than ten RTL modules; remaining work includes RoPE ROM generation, golden-model numerical verification, a weight-loading handshake, and full end-to-end simulation.

***Index Terms*—** *Self-attention, transformer accelerator, RTL, SystemVerilog, bfloat16, RoPE, Grouped Query Attention, systolic array, FPGA.*

I. INTRODUCTION

Transformer-based models rely on the self-attention mechanism, which is computationally dominated by matrix multiplications and a row-wise softmax normalization. Mapping this mechanism to dedicated hardware requires careful management of arithmetic precision, data movement, and resource reuse. This work implements an end-to-end streaming attention core that takes a token vector through Q/K/V projection, RoPE positional encoding, GQA head mapping, $Q \cdot K^T$ similarity computation, scaling and masking, softmax normalization, and finally the score·V product, all communicated between modules using an AXI-Stream-like valid/ready handshake.

A central design goal was resource efficiency. Rather than instantiating separate hardware for the two large matrix-multiplication stages of attention, the design shares a single systolic PE array — referred to in this report as Module A — between the $Q \cdot K^T$ computation and the score·V computation, distinguishing the two data streams with a tag bit and arbitrating access at the input.

II. SYSTEM ARCHITECTURE

Fig. 1 shows the top-level data flow. Input tokens arrive over the AXI interface and are written into a double-buffered memory (dbuf), which allows new data to be loaded while computation proceeds on the current buffer. Data then passes sequentially through QKV projection, RoPE, and GQA head mapping. In the second stage, the $Q \cdot K^T$ array produces similarity scores, which are scaled, masked, and normalized by the softmax unit before the score·V operation produces the final output, which is returned to the AXI interface. As noted above, the $Q \cdot K^T$ array and the score·V unit share the same physical MAC hardware (Module A); an arbiter/router pair manages access to this shared resource using a one-bit tag carried alongside the data.

Fig. 1. YAE top-level data flow

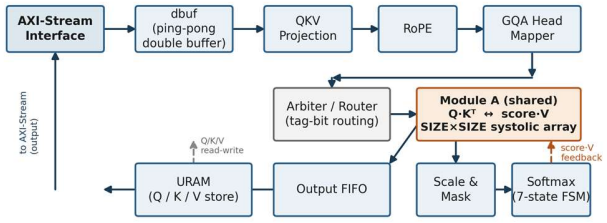

Fig. 1. YAE top-level data flow: AXI-Stream input, double-buffered memory, QKV/RoPE/GQA front end, and the shared Module A ($Q \cdot K^T$ / score·V) with its arbiter and router.

III. BF16 ARITHMETIC CORE

All datapath computation uses the 16-bit bfloat16 (BF16) format, which reduces memory footprint and interconnect bandwidth relative to IEEE-754 float32 while retaining float32's dynamic range. The arithmetic core provides four basic units: an adder, a multiplier, and two format converters. Both the adder and multiplier are implemented as two-stage pipelines rather than single-cycle combinational blocks, so every module that consumes their results is built around the valid/ready handshake instead of a fixed latency assumption. To support the shared-hardware scheme described in Section II, all data on the internal bus is carried as 17 bits: a 16-bit BF16 value plus a 1-bit tag identifying the data's source stream.

IV. QKV PROJECTION

The QKV projection module performs a matrix-vector multiply-accumulate between the incoming token vector and a weight matrix. A single parameterized module is instantiated four times — for the Q, K, V, and output projections — each configured with different weights. Because the design supports GQA, the output dimensions of the K and V projections are configured to be smaller than that of Q, reflecting the smaller number of key/value heads relative to query heads.

V. ROTARY POSITIONAL ENCODING (ROPE)

RoPE embeds positional information into the Q and K vectors by applying a position-dependent rotation using sine and cosine coefficients. The implementation uses the "rotate-half" pairing convention, in which element i is paired with element i + D_MODEL/2 rather than with an adjacent element. This pairing choice is currently an assumption rather than a confirmed specification, since it was not made explicit in the project plan, and it is flagged for verification once a golden-model reference script is available (see Section IX).

VI. GROUPED QUERY ATTENTION (GQA) MAPPER

In Grouped Query Attention, the number of query heads exceeds the number of key/value heads, and groups of query heads share a common K/V pair — in the current configuration, eight Q heads map to two KV heads. The GQA mapper does not move or duplicate data; it generates the index that determines which KV head a given Q head should use, deferring the actual head selection to downstream modules.

VII. SHARED Q·K^T / SCORE·V SYSTOLIC ARRAY

Module A is a SIZE × SIZE output-stationary systolic PE array. Query and key elements are streamed in serially along the array's depth dimension; once all elements for a given tile have been accumulated, the array switches to a parallel read-out phase. Because this array is time-shared between the Q·K^T similarity computation and the score·V reduction, an input arbiter governs access: priority is given to score·V requests arriving from the softmax stage, with new Q·K^T inputs accepted afterward. This ordering keeps the softmax-to-output path from stalling behind newly arriving queries.

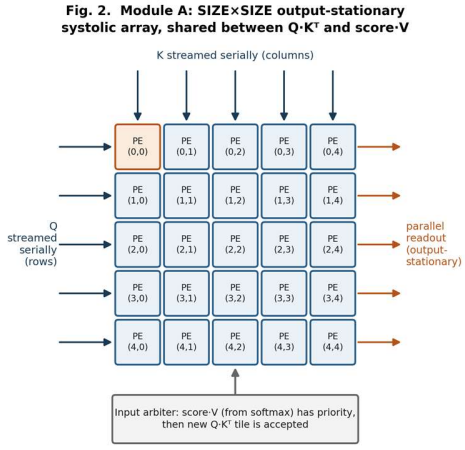

Fig. 2. Module A: SIZE×SIZE output-stationary systolic PE array, showing serial Q/K streaming, parallel output-stationary readout, and the input arbiter shared between Q·K^T and score·V.

VIII. SCALING, MASKING, AND SOFTMAX

Raw similarity scores are scaled, and masked positions are set to negative infinity within a three-stage pipeline governed by a backpressure chain. Because softmax requires a row-wise sum, its elements are processed sequentially through a seven-state finite state machine covering exponentiation, summation, reciprocal computation, and normalization. This stage was designed to prioritize numerical accuracy over maximum throughput.

IX. MEMORY AND DATA ROUTING

The supporting infrastructure consists of the dbuf double-buffer, which allows new input data to be loaded while computation continues on the current tile; URAM-based memories that store the Q, K, and V vectors; and an output FIFO that transfers completed results to the AXI interface. The arbiters and routers that connect these blocks direct traffic on the shared hardware to its correct destination using the tag bit introduced in Section III.

X. OPEN ISSUES AND ASSUMPTİONS

Five items are currently flagged in code comments as pending team approval or verification:

- The RoPE rotate-half pairing rule (Section V).
- Missing sine/cosine ROM files required by RoPE.
- Whether GQA head mapping will be realized physically or index-based.
- The assumed interface behavior between softmax and its neighboring modules.
- Absence of a locking mechanism in the weight-loading process.

XI. CONCLUSION AND FUTURE WORK

The design presented here implements more than ten RTL modules covering the full self-attention pipeline, from projection through RoPE, GQA mapping, similarity computation, softmax, and output generation, with a shared systolic array reducing MAC resource duplication. Planned next steps are: generating the RoPE ROM files, performing numerical verification against a golden software model, adding a handshake to the weight-loading path, and running full end-to-end pipeline simulation.

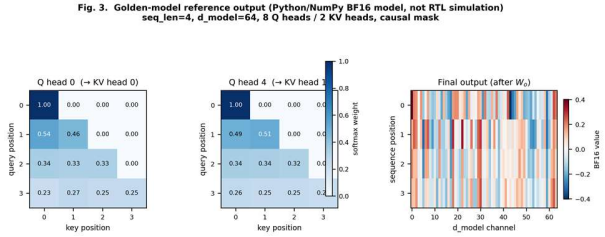

Fig. 3. Golden-model reference output for a representative test case (seq_len=4, d_model=64, 8 Q / 2 KV heads, causal mask): softmax attention weights for two Q heads, and the final projected output. From the Python/BF16 golden model; pending replacement with matching RTL results.

REFERENCES

[1] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017.

[2] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv:2104.09864, 2021.

[3] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," arXiv:2305.13245, 2023.

[4] Google Brain, "bfloat16: The secret to high performance on Cloud TPUs," Google Cloud Blog, 2019.